

JavaScript

For non-JavaScript developers

Brief history of JavaScript

Netscape Navigator & Brendan Eich

In 1995 Netscape tasked Brendan Eich to create a new dynamic language for the Web.

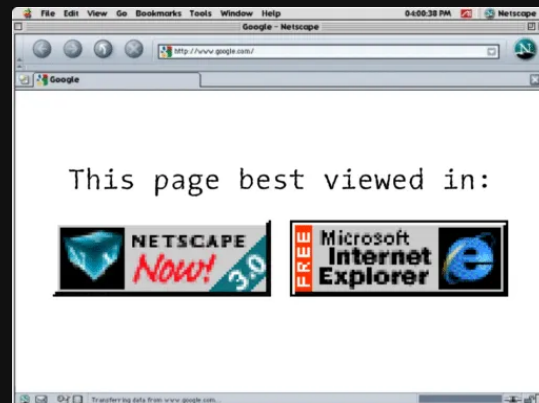
10 days later LiveScript is born.

As a collaboration with Sun Microsystems, LiveScript is renamed JavaScript as a complement to Java for web development.

LiveScript 

Netscape vs Microsoft

Microsoft reverse engineers JavaScript, creates JScript for Internet Explorer 4.



ECMA

In 1997 as Netscape quickly loses market shares, Brendan submits "JavaScript" to ECMA. Without copyright ownership of the name, JavaScript gets renamed as ECMAScript.



The first golden age

ECMAScript sees its first golden age as Ajax, a set of technologies for JavaScript and XML, gets released.

JQuery, MooTools and many other libraries are released during this period.



Google arrives just in time

In 2008 Google releases Chrome, using a new engine named V8, adds JiT and forces the work on ECMAScript to resume after Microsoft blockade, ES5 finally sees the light of day.



Ryan Dahl announces Node

Enters JSConf 2009.

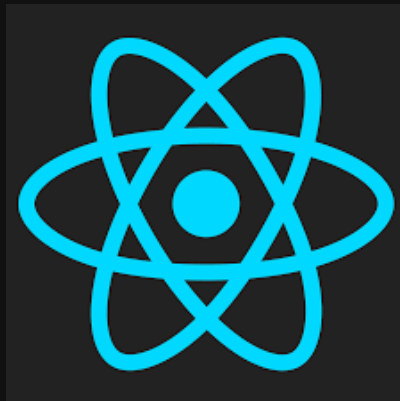
Announces Node.

Leaves.



Second golden age: React

In 2013 Facebook revolutionizes the web dev world by creating an endless amount of new jobs to fix React applications.



What is JiT?

Is JavaScript a compiled or interpreted language?

Is JavaScript a **compiled** or interpreted language?

Is JavaScript a **compiled** or **interpreted** language?

JavaScript uses **Just in Time (JiT)** compilation,
meaning it is compiled during execution.

JiT in practice

Compiler (TypeScript lives here!)

Scope

Engine

Compiler (TypeScript lives here!)

Generates & hoist code

Scope

Engine

Compiler (TypeScript lives here!)

Generates & hoist code

Scope

Maintains look-up lists

Engine

Compiler (TypeScript lives here!)

Generates & hoist code

Scope

Maintains look-up lists

Engine

Executes code

JavaScript is a **dynamic language** because variables are assigned by the engine during runtime based on the value they hold at the time.

Compiler

```
1 var a = 1;  
2  
3 function myFunc(b) {  
4   console.log(a + b)  
5 }  
6  
7 myFunc(2)
```

How many declarations are in this snippet?

Compiler

```
1 var a = 1;  
2  
3 function myFunc(b) {  
4   console.log(a + b)  
5 }  
6  
7 myFunc(2)
```

a is declared in the global scope.

Compiler

```
1 var a = 1;  
2  
3 function myFunc(b) {  
4   console.log(a + b)  
5 }  
6  
7 myFunc(2)
```

myFunc is declared in the global scope.

Compiler

```
1 var a = 1;  
2  
3 function myFunc(b) {  
4   console.log(a + b)  
5 }  
6  
7 myFunc(2)
```

b is declared in myFunc scope.

The compiler's job is to **define** variables and function and **hoist** them at the beginning of the relative scope.

The engine performs mainly 2 jobs:

- LHS lookup
- RHS lookup

The engine performs mainly 2 jobs:

- LHS lookup assigns a value
- RHS lookup

The engine performs mainly 2 jobs:

- **LHS** lookup assigns a value
- **RHS** lookup retrieves a value

Engine

```
1 var a = 1;  
2  
3 function myFunc(b) {  
4   console.log(a + b)  
5 }  
6  
7 myFunc(2)
```

LHS lookups: **a**, **b** (implicit)

Engine

```
1 var a = 1;  
2  
3 function myFunc(b) {  
4   console.log(a + b)  
5 }  
6  
7 myFunc(2)
```

RHS lookups: *a*, *myFunc(2)*, *a +*, *+ b*

Errors:

- **Reference Error** is a failure of LHS or RHS lookup, the variable does not exist at the time of lookup.
- **Type Error** is operation failure on an value found in scope.

Scope

Scope is an author defined environment where code is evaluated and executed.

```
1 var a = 1;
2
3 function myFunc(b) {
4     function myOtherFunc(c) {
5         console.log(a, b, c)
6     }
7     myOtherFunc(3)
8 }
9
10 myFunc(2) // 1, 2, 3
```

How many variables does each scope have?

```
1 var a = 1;  
2  
3 function myFunc(b) {  
4   function myOtherFunc(c) {  
5     console.log(a, b, c)  
6   }  
7   myOtherFunc(3)  
8 }  
9  
10 myFunc(2) // 1, 2, 3
```

3 scopes, each has one variable

Scope lookup stops once it finds a matching variable

```
1 var a = 1; // shadowed by the other a
2
3 function myFunc() {
4   var a = 2
5   console.log(a)
6 }
7
8 myFunc() // 2
```

Principle of least privilege and scope

Function scope

The "default" JS scope works for **function blocks** only. This is bad.

Examples of scope pollution

```
1  for (var i = 0; i < 10; i++) { // false scope
2    console.log(i)
3  }
4
5  if (true) {
6    var b = 20; // false scope
7    function f() { console.log(1) } // false scope
8  }
9
10 console.log(i, b) // 10, 20
11 f(); // 1
```

The solutions exist, and they called **let** and **const**

Block scoped

```
1  for (let i = 0; i < 10; i++) {  
2    console.log(i)  
3  }  
4  
5  if (true) {  
6    const b = 20;  
7    const f = () => { console.log(1) }  
8  }  
9  
10 console.log(i, b) // Reference Error  
11 f(); // Reference Error
```

Advantages of block scope:

- Automatic garbage collection
- Least privilege by default
- Const introduces fixed values
- Const prevents name reassignment

Interlude #1

Always name your functions, you don't want to be the dev debugging this.

Task		
Timer fired		
Function call		
(anonymous)		
t.completeTask		
(anonymous)		
\$digest		
(anonymous)	On ignore list (\\node_modules\\ ^node:)	(anonymous)
(anonymous)		fn
(anonymous)		get
(anonymous)		
(anonymous)		
open	send	

```
1 setTimeout(() => { // bad
2   console.log(1)
3 }, 1000)
4
5 const a = () => { console.log(1) }
6 setTimeout(a, 1000) // good
```

this Keyword

this is a special reference that is not defined by the compiler, but the engine. Its value depends on the place that invoked it.

```
1 function greetings() {  
2   console.log("Hello I am", this.name);  
3 }  
4  
5 var me = {  
6   name: "Francesco"  
7 }  
8 var bossman = {  
9   name: "Vlad"  
10 }  
11  
12 greetings.call(me) // Hello I am Francesco  
13 greetings.call(bossman) // Hello I am Vlad
```

Primitives

Primitives:

- string
- number (NaN is a number!)
- boolean
- undefined
- null
- symbol
- bigint

Primitives are **immutable**, their *values* cannot be altered, every new value is individually stored in memory.

```
1 let a = 1;
2 let b = a;
3
4 b = 2;
5
6 console.log(a) // 1
7 console.log(b) // 2
```

JavaScript automatically uses object constructors behind the curtains when performing operations on primitives.

```
1 let realString = "a"
2 typeof realString // "string"
3 realString instanceof String // false
4
5 let objectString = new String("b")
6 typeof objectString // "object"
7 objectString instanceof String // true
```

Never use object constructors to define a primitive.

undefined is a special primitive, it implicitly represents the absence of a value.

```
1 let a; // undefined
2
3 const b = () => {
4   return; // undefined
5 }
6
7 let c = [1,2].find(n => n === 3) // undefined
```

- **undefined** should not be used explicitly, but only detected implicitly.
- **null** can be used in cases where we explicitly don't want a value to be returned.
- **null** and **undefined** have no constructor.

In an average interaction, the difference between **null** and **undefined** should not matter.

More generic falsy checks are safer when the type difference does not matter.

```
1 let obj = { a: 1, b: null };
2
3 typeof a.b === "undefined" // false
4 typeof a.c === "undefined" // true
5
6 !a.b // true
7 !a.c // true
```

Objects

Everything that is *not* a primitive is an exotic object, some examples:

- Array
- Function
- Error
- Date
- Set
- Proxy
- Reflect
- Map
- Math
- ...

Objects are not stored as immutable values, but as reference values. Meaning all variables are connected to the same value and **no copies are created.**

```
1 let a = [1,2];  
2 let b = a;  
3  
4 b.push(3);  
5  
6 console.log(a) // [1,2,3]  
7 console.log(b) // [1,2,3]
```

Class inheritance and polymorphism (OOP) as seen in other languages are not concepts that JavaScript inherits.

JavaScript uses prototypal inheritance, meaning each object has a prototype property, which point to another object (or null!) recursively. Objects inherit from other objects in a dynamic rather than static way.

Java inheritance

```
1 class Animal {
2     void bark() {
3         System.out.println("ham ham");
4     }
5 }
6
7 class Dog extends Animal {
8     void eat() {
9         System.out.println("gnam gnam");
10    }
11 }
12
13 public class Main {
14     public static void main(String[] args) {
15         Dog dog = new Dog();
16     }
17 }
```

JavaScript prototypal inheritance

```
1  const animal = {  
2    bark: () => console.log('ham ham')  
3  };  
4  
5  const dog = Object.create(animal);  
6  dog.eat = () => console.log('gnam gnam');  
7  
8  dog.eat(); // Own method  
9  dog.bark(); // Inherited method  
10  
11 animal.bark = () => console.log('bau bau')  
12  
13 dog.bark(); // bau bau
```

JavaScript prototypal inheritance, class syntax

```
1 class Animal {  
2   bark() { console.log('ham ham'); }  
3 };  
4  
5 class Dog extends Animal {  
6   eat() { console.log('gnam gnam'); }  
7 };  
8  
9 const myDog = new Dog();  
10 myDog.eat(); // Own method  
11 myDog.bark(); // Inherited method  
12  
13 Animal.prototype.bark = () => console.log('bau bau');  
14 myDog.bark(); // bau bau
```

This works because 'class' is just syntactic sugar for the prototype system.

Trying to emulate classical inheritance in JavaScript tends to generate hard to understand code and hard to debug problems linked to the way reference works.

Our purpose when writing JavaScript code is not to copy behaviour from other languages, but use JavaScript's own features to solve the unique problems encountered in a frontend environment.

Interlude #2

This is called **callback hell**

```
1  fetchItalianJoke((joke) => {  
2    // if (...)  
3    translateJokeToRomanian(joke, (translateJoke) => {  
4      // if (...)  
5      doubleCheckTranslation(translateJoke, (correctTranslatedJoke) =>  
6        // if (...)  
7        postJoke(correctTranslatedJoke, () => {  
8          // if (...)  
9          console.log("Done!")  
10       })  
11     })  
12   })  
13 })
```

I have omitted reject/error for readability purposes.

Number of scopes increases drastically as nesting gets deeper. Promises return a similar result.

This is the same code, using **async/await**

```
1  const getJoke = async () => {  
2    try {  
3      const joke = await fetchItalianJoke()  
4      const translatedJoke = await translateJokeToRomanian(joke)  
5      const correctTranslatedJoke = await doubleCheckTranslation(trans  
6  
7      await postJoke(correctTranslatedJoke)  
8      console.log("Done!")  
9    } catch (err) { // err is block scoped!  
10     console.error(err)  
11   }  
12 }
```

3 Block scopes, adding 10 more awaits will still max out at 3 block scopes.

Pure Functions

Pure functions are functions devoid of side effects, making them more reliable and easier to understand.

```
1  const num1 = 1;
2  const num2 = 2;
3
4  const sum = (a,b) => a + b;
5
6  const num3 = sum(num1, num2)
7
8  console.log(num1) // 1
9  console.log(num2) // 2
10 console.log(num3) // 3
```

Idempotency (same input -> same output) create predictable and easily testable code regardless of how many times it is executed.

```
1 for (let i = 0; i < 100; i++) {  
2   // run this test 100 times  
3   test('adding 2 numbers results in their sum', () => {  
4     const r1 = Math.floor(Math.random() * 10) + 1 // 1-10  
5     const r2 = Math.floor(Math.random() * 10) + 1 // 1-10  
6  
7     expect(sum(r1, r2)).toBe(r1 + r2);  
8   });  
9 }
```

Refactoring code to pure functions can lead to more modular code as well.

```
1  const addNum = (arr) => { // impure
2    arr.push(arr.length + 1)
3  }
4
5  let numbers = [1,2,3,4]
6  addNum(numbers) // side effect on numbers
```

```
1  const addNum = (arr) => { // pure
2    return arr.length + 1
3  }
4
5  let numbers = [1,2,3,4]
6  let number = addNum(numbers) // no side effects
```

Reducing side effects in an application state can reduce the occurrence of unexpected bugs, raising confidence and readability in the code.

Immutability

While purity allows us to change our state only once, **immutability** allows us to not change it at all.

JavaScript primitives are immutable, but objects are not.

```
1 const a = 1
2 a = 2 // TypeError!
3
4 const b = { a: 1 }
5 b.a = 2 // { a: 2 }
```

Both JavaScript and TypeScript provide utilities to make objects readonly, but they are only shallow.

```
1  const x = Object.freeze({ a: 1, b: { c: 2 } })
2
3  x.a = 2 // error
4  x.b.c = 1 // allowed
```

```
1  const x: Readonly<any> = { a: 1, b: { c: 2 } };
2
3  x.a = 2 // error
4  x.b.c = 1 // allowed
```

Immutability is achieved by discarding old values and creating new ones to keep our state up to date, never mutating existing values.

```
1  const updateUserRole = (user, role) => {  
2    const newUser = { ...user, role }  
3  
4    return newUser  
5  }
```

While there are tricks and libraries that allow deep immutability, the issue is in our way of thinking about our code, more than the tools we use.

You should treat all data as immutable by default.

Single source of truth

There's only one value, everything else is a reference to it.

By having a centralized application state, the UI needs only to listen to a single entity, and update only when that entity is updated.

This greatly simplifies how a UI is written and brings down maintenance time considerably.

An immutable, single source of truth store, using pure functions for updates.

```
1  const reducer = (state, { type } = {}) => {
2    switch(type) {
3      case 'increment':
4        return { value: state.value + 1 }
5      default:
6        return state
7    }
8  }
9
10 const store = {
11   state: { value: 1 },
12   dispatch: function (action) {
13     this.state = reducer(this.state, action)
14   }
15 }
```

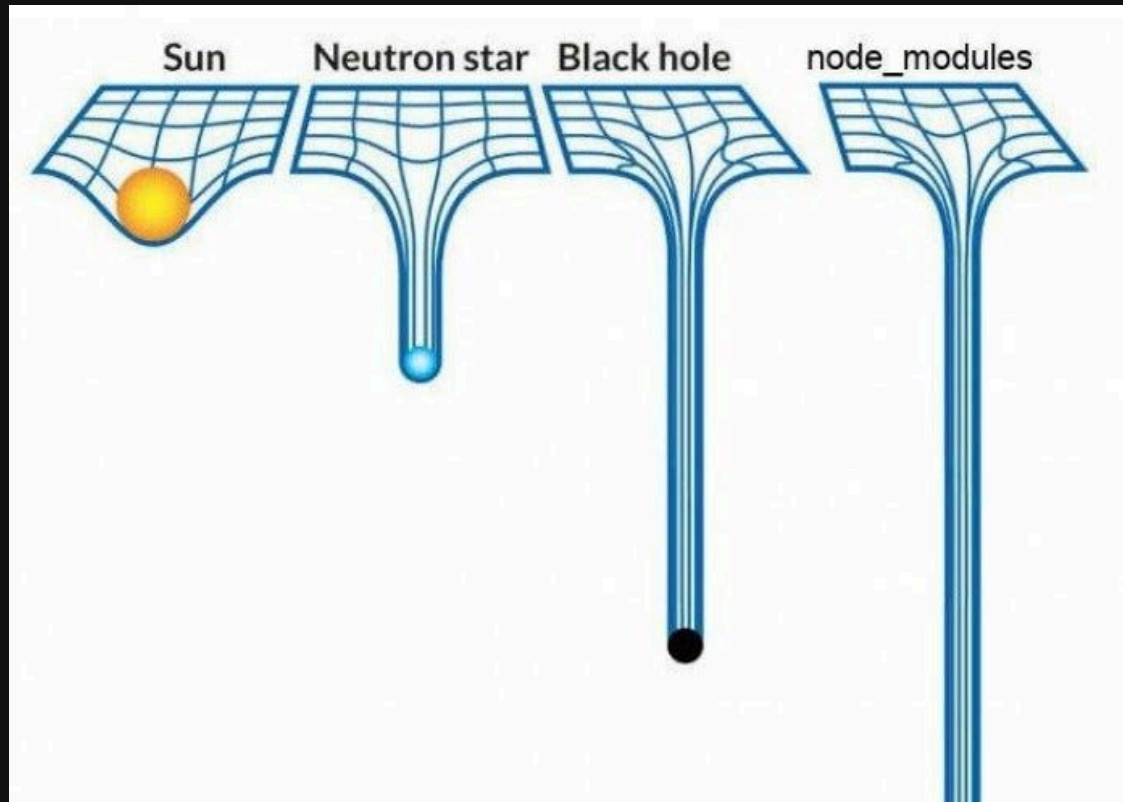

Making it immutable by hiding the state and exposing a frozen object to views.

```
1  const stateSymbol = Symbol("state");
2
3  const store = {
4    [stateSymbol]: { value: 1 },
5    getState: function () {
6      return Object.freeze(this[stateSymbol])
7    },
8    dispatch: function (action) {
9      this[stateSymbol] = reducer(this[stateSymbol], action)
10   }
11 }
```

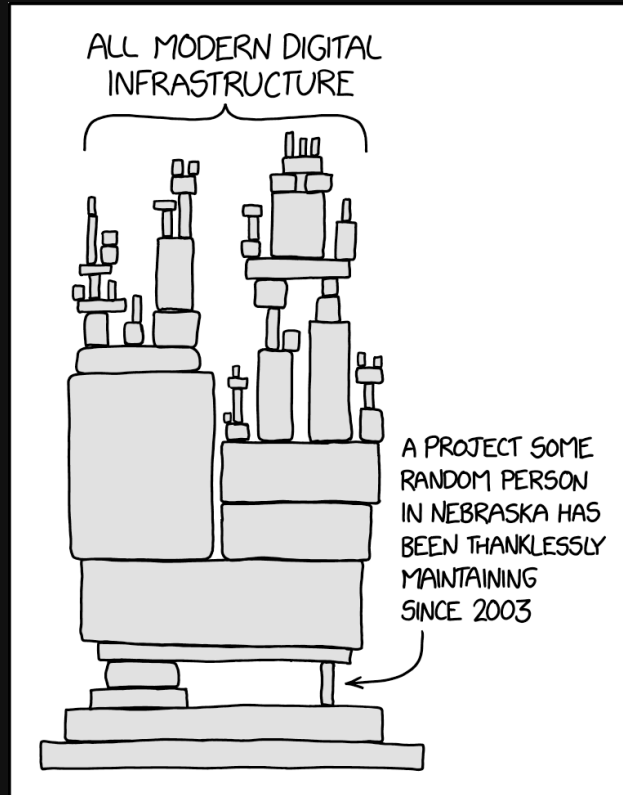
The node package manager

The JavaScript ecosystem is built around **small, user created modular libraries**. The ecosystem exploded during the last decades, with more than 2 million packages currently hosted on npm.

npm packages are just JavaScript project, and as any project, they can import other projects. This can lead to immense dependency trees usually multiple GBs in size.



The user-created nature and deeply nested structure of npm modules can lead to structural issues in even the best curated codebases.



The left-pad incident

- In 2016 kik asks npm the rights over the kik package owned by Azer Koçulu.
- npm agrees, removing the ownership from Koçulu.
- Koçulu removes his 273 packages from npm, including left-pad.
- Widespread installation failures for tools like Babel halt development for countless companies.
- Major companies like Meta, Netflix, Spotify and Paypal are affected.
- npm restores left-pad and tightens rules around unpublishing packages with dependents.

Unlike Maven for Java, npm relies on open source contributions.

This makes the ecosystem wider, but increases the attack surface for supply chain attacks.

Libraries and the evolution of JS

The contributions from the open source community cannot be understated for the growth of JavaScript.

Many features like **Promises** (bluebird), the **fetch API** (jQuery), **.map** and **.filter**, (lodash) and even the **import/export** syntax (node) were inspired by open source libraries.

Avoiding external libraries is not the point, knowing the features of the language is.

Should I use a library?

- Is the problem already solved by modern JavaScript?
- Does the library solve a complex problem?
- Is the library healthy?

Should I use a library?

- Is the problem already solved by modern JavaScript?

Check the [MDN Docs](#).

- Does the library solve a complex problem?
- Is the library healthy?

Should I use a library?

- Is the problem already solved by modern JavaScript?

Check the [MDN Docs](#).

- Does the library solve a complex problem?

[Complex animations](#) or [date manipulation](#) are examples of complex problems.

- Is the library healthy?

Should I use a library?

- Is the problem already solved by modern JavaScript?

Check the [MDN Docs](#).

- Does the library solve a complex problem?

[Complex animations](#) or [date manipulation](#) are examples of complex problems.

- Is the library healthy?

Check [GitHub](#) and [npm](#) to assess the security risk and maintenance burden.

Code you don't understand is code you can't trust.

If you need to execute it to understand it, there's a problem with the code.